Plan implementacije: Sequence Alignment i Read Mapping

1. Cilj i obim projekta

Cilj projekta je implementirati Smith-Waterman algoritam (lokalno poravnanje sekvenci s linearnom kaznom za praznine) i na njemu zasnovan jednostavan read mapper, a zatim uporediti serijsku i paralelnu verziju. Težište je na paralelizaciji i mjerenju performansi.

U obimu:

- serijski Smith-Waterman s linearnom kaznom za praznine, tracebackom i CIGAR zapisom
- jednostavan k-mer indeks (sortiran niz) referentnog genoma, seeding i extension
- paralelizacija po readovima
- mjerenje ubrzanja, efikasnosti i GCUPS-a

2. Tehnologije (prijedlog)

Dio	Alat
Jezik	C++17, `g++ -O2`
Paralelizacija na CPU	OpenMP
GPU	CUDA
Verifikacija	Biopython `PairwiseAligner` ili EMBOSS `water`
Generisanje podataka	vlastiti generator ili `wgsim`
Mjerenje	`std::chrono`, `perf`/`gprof` za profiliranje

3. Faze rada

Faza 1: serijski Smith-Waterman

- Rekurzija s jednom matricom H i linearnom kaznom praznine W_k = e·k (kazna po znaku praznine).
- Traceback od maksimuma do nule, ispis poravnanja i CIGAR-a.
- Brža verzija koja čuva samo dva reda tabele i daje samo skor. Koristi se za mjerenje brzine.

Faza 2: korektnost

- Testovi na malim ručno provjerenim slučajevima.
- Poređenje s Biopythonom/EMBOSS-om na nasumičnim parovima.
- Samoprovjera: full verzija i score-only verzija moraju dati isti skor.
- **Izlaz:** serijska verzija je "zlatni standard" za sve kasnije provjere.

Faza 3: read mapping

- Učitavanje referentnog genoma i readova (FASTA/FASTQ).
- k-mer indeks (npr. k = 12 do 15): sortiran niz parova (k-mer, pozicija). K-meri koji se previše ponavljaju (npr. više od 100 puta) se preskaču.
- Seeding: izdvajanje k-mera iz reada, pretraga indeksa binarnom pretragom, glasanje za kandidat-pozicije po dijagonali (p − offset), uz malu toleranciju zbog praznina.
- Filtriranje: zadržavaju se samo kandidati s dovoljno glasova (npr. najboljih 5).
- Extension: Smith-Waterman nad regijom reference oko kandidata (dužina reada plus margina s obje strane).
- Izbor najboljeg kandidata, ispis pozicije, skora i CIGAR-a. Readovi bez kandidata označavaju se kao nemapirani.
- **Izlaz:** tačnost mapiranja na sintetičkim podacima (poznata prava pozicija).

Faza 4: profiliranje

- Izmjeriti udio vremena: učitavanje, indeksiranje, seeding, poravnanje, ispis.
- Rezultati mjerenja pokazuju koji dio programa najviše traje i šta se isplati paralelizirati.

Faza 5: paralelizacija

1. Po readovima – OpenMP: readovi se dijele između niti procesora (#pragma omp parallel for schedule(dynamic)). Svaka nit obrađuje svoje readove i ima vlastite bafere.
2. Poređenje**:** Serijsku i OpenMP verziju pokrećemo na istim readovima i s istim parametrima. Poredimo vrijeme izvršavanja i provjeravamo da svaki read ima isti skor i pronađenu poziciju u obje verzije.
3. CUDA: Obraditi veći broj nezavisnih parova (read, kandidatska regija genoma) na GPU-u, u grupama. Rezultate uporediti sa serijskom verzijom.

Faza 6: evaluacija i izvještaj

- Jaka skalabilnost: fiksan skup podataka, broj niti 1, 2, 4, 8, ... (za CUDA se mijenja broj readova po grupi).
- Slaba skalabilnost: podaci rastu proporcionalno broju niti.
- Analiza uskih grla: Amdahlov zakon, memorijski pojas, balansiranje opterećenja.
- Pisanje rada i priprema prezentacije.

4. Test podaci

Skup	Opis	Namjena
Mali	Demo primjer TGTTACGG / GGTTGACTA (8 i 9 baza)	Provjera korektnosti

Skup	Opis	Namjena
Sintetički	Nasumična referenca (npr. 1 do 5 Mb) i readovi dužine 100 do 150 s ubačenim zamjenama i indelima	Tačnost i skalabilnost
Stvarni	Genom bakterije (npr. E. coli, oko 4,6 Mb) i simulirani readovi	Realističnija evaluacija

Sintetički podaci omogućavaju računanje tačnosti jer je prava pozicija svakog reada poznata.

5. Metrike

- **Vrijeme izvršavanja:** mjerimo serijsku verziju, OpenMP verziju s p nitima i CUDA verziju s grupom veličine b, uvijek na istim readovima.
- **Ubrzanje OpenMP verzije:** $S_{OpenMP}(p) = T_{serijski}/T_{OpenMP}(p)$.
- **Efikasnost OpenMP verzije:** $E_{OpenMP}(p) = S_{OpenMP}(p)/p$.
- **Ubrzanje CUDA verzije:** $S_{CUDA}(b) = T_{serijski}/T_{CUDA}(b)$. Vrijeme CUDA obrade mjerimo za različite veličine grupe; vrijeme slanja podataka na GPU bilježimo posebno ili ga uključimo u ukupno vrijeme.
- **GCUPS:** broj stvarno izračunatih ćelija svih Smith–Waterman poravnanja podijeljen vremenom poravnanja i sa 10^9.
- **Tačnost mapiranja:** udio readova pronađenih na pravoj poziciji, uz toleranciju od nekoliko baza.

Svako mjerenje ponoviti više puta (npr. 5) i uzeti medijanu.

6. Rizici i mjere

Rizik	Mjera
Paralelna verzija daje drugačije rezultate	Automatsko poređenje sa serijskom verzijom na istim ulazima
Matrica zauzima previše memorije	Verzija koja računa samo skor (bez cijele matrice)

7. Kriteriji završetka

- ☐ Rezultati se poklapaju s referentnim alatom (Biopython) na nasumičnim parovima
- ☐ Mapper ispravno pronalazi sintetičke readove (tačnost iznad dogovorenog praga)
- ☐ OpenMP verzija po readovima s izmjerenim ubrzanjem
- ☐ CUDA verzija po readovima s izmjerenim ubrzanjem
- ☐ Grafovi ubrzanja, efikasnosti i GCUPS-a
- ☐ Napisan rad i pripremljena prezentacija